

Compiladors: Examen parcial de laboratori.

8 d'abril de 2025

ATENCIÓ: Al Racó trobareu els jocs de proves i codi necessari per a fer l'examen. El paquet conté versions modificades dels mòduls `common/SemErrors.*` i `common/TypesMgr.*` amb els mètodes necessaris per donar els nous errors o gestionar els nous tipus que apareixen als exercicis de l'examen. **ABANS DE COMENÇAR A FER RES**, llegiu les instruccions del final de l'enunciat per veure com descarregar-lo i instal·lar-lo.

ATENCIÓ: Cal entregar l'examen en un fitxer `.tgz` pujat al Racó. **Llegiu les instruccions del final de l'enunciat per veure com generar-lo.**

PUNTUACIÓ: Els tres primers punts de la nota de laboratori s'obtenen amb els jocs de proves de la pràctica base. La resta s'obtenen superant els jocs de proves específics de l'examen. La correcció és **automàtica**, a través dels jocs de proves d'aquest enunciat, més un conjunt addicional de jocs de proves privats.

IMPORTANT: L'examen consta de dos exercicis independents. Podeu fer-los en qualsevol ordre. Es recomana fer cada exercici incrementalment, resolent cada joc de proves abans de passar al següent.

1 Funcions amb tipus de retorn any (3 punts)

Volem afegir a l'ASL la possibilitat de tenir funcions que retornin un tipus qualsevol, cosa que expressarem amb el nou tipus `any`. El codi que cridi a la funció haurà de fer un *cast* explícit al tipus desitjat.

Les condicions d'ús d'aquesta nova funcionalitat del llenguatge són les següents:

- El tipus `any` només pot aparèixer com a tipus de retorn en la declaració d'una funció. És a dir, no es poden declarar variables ni paràmetres de tipus `any`.
- Per poder usar el resultat d'una funció `any` en una expressió, cal fer-li un *cast* explícit al tipus desitjat usant l'operació `anycast` (descrita més endavant). Altrament, es produirà un error, ja que `any` no es compatible amb cap altre tipus ni amb cap operació.
- El resultat d'una funció `any` només es pot convertir a tipus bàsics.

Un exemple de codi:

```
1 func f(a:int, b:bool, x:array[10] of char) : any
2   if a > 10 then
3     return b or not (x[1] == 'Y');
4   else if not b then
5     return x[a];
6   else
7     return a*2.0;
8   endif
9   endif
10 endfunc
11
12 func main()
13   var i,j : int
14   var z : float
15   var a,b : array [10] of char
16
17   i = 5;
18   j = 2;
19   z = i+j;
20   z = anycast<float>(f(i+1, z<22, a));
21   if anycast<bool>(f(15, false, a)) then
22     a[3] = anycast<char>(f(j, j>=100, a));
23   endif
24 endfunc
```

Joc de proves 1 (1 punt). El primer pas és ampliar la gramàtica amb el nou tipus `any`. Afegiu-lo com a *basic_type*, cosa que us simplificarà la feina. Haureu de modificar el *SymbolsVisitor* per que es crei adequadament el tipus `any` i es decori el node corresponent de l'arbre.

Al `common` modificat per a l'examen trobareu que la classe `TypesMgr` disposa de mètodes per crear i tractar el tipus `any`. Els mètodes `equalTypes`, `comparableTypes` i `copyableTypes` han estat adaptats per funcionar amb el nou tipus `any`.

Amb això haurieu de passar el primer joc de proves:

El codi següent:

```
1 func f(a:int, b:bool, x:array[10] of char) : any
2   if a > 10 then
3     return b or not x[1];
4   else if not b then
5     return x[a];
6   else
7     return a*2.0;
8   endif
9 endif
10 endfunc
11
12 func main()
13   var i,j : int
14   var z : float
15   var a,b : array [10] of char
16
17   i = 5;
18   j = a[i] or b>0;
19   z = i+j;
20   // f retorna un tipus any, i sense fer anycast la suma dona error:
21   z = f(i+1, z<22, a) + 2;
22
23   // tampoc es fa cast del resultat de f:
24   if j>i or f(15, false, a) and not b then
25     // assignacio sense cast:
26     a[3] = f(j, j>=100, b);
27   endif
28   f(5, true, b);
29   write z;
30 endfunc
```

genera els errors:

```
Line 3:17 error: Operator 'not' with incompatible types.
Line 18:4 error: Assignment with incompatible types.
Line 18:11 error: Operator 'or' with incompatible types.
Line 18:15 error: Operator '>' with incompatible types.
Line 21:22 error: Operator '+' with incompatible types.
Line 24:28 error: Operator 'and' with incompatible types.
Line 24:32 error: Operator 'not' with incompatible types.
Line 26:10 error: Assignment with incompatible types.
```

Joc de proves 2 (1 punt). A continuació afegirem l'operació de *cast* que converteix el resultat d'una funció *any* en el tipus desitjat.

La sintaxi de l'operació és: `anycast<T>(expr)`, on *T* és el tipus basic al qual cal convertir el resultat de la funció, i *expr* és la crida a la funció.

Us caldrà modificar la gramàtica per afegir `anycast` a les expressions. També caldrà modificar *TypeCheckVisitor* per validar que el tipus de l'expressió argument és *any*. Trobareu un nou mètode per generar aquest error a la versió modificada de *SemErrors*.

Així, el segon joc de proves:

```
1 func g() : bool
2     return true;
3 endfunc
4
5 func f(a:int, b:bool, x:array[10] of char) : any
6     if a > 10 then
7         return b or not x[1];
8     else if not b then
9         return x[a];
10    else
11        return a*2.0;
12    endif
13 endif
14 endfunc
15
16 func main()
17     var i,j : int
18     var x,z : float
19     var a,b : array [10] of char
20     var p : bool
21     i = 5;
22     j = a[i] or b>0;
23     z = i+j;
24     // la funcio f retorna any:
25     z = anycast<float>(f(i+1, z<22, a)) + 2;
26     // la suma de any i float dona un float, que no es any:
27     x = anycast<float>(f(j+2, z<50, b) + 5.2);
28     // la funcio g no retorna any, i a[3] no es de tipus any:
29     x = anycast<float>(g(23,2)) + anycast<int>(a[3]);
30
31     if j>i or anycast<bool>(f(15, false, a)) and not b then
32         a[3] = anycast<char>(f(j, j>=100, b));
33         b = anycast<bool>(f(j, j>=100, b) and not p) or x!=2;
34     endif
35     f(5, true, b);
36     write z;
37 endfunc
```

genera els errors:

```
Line 7:17 error: Operator 'not' with incompatible types.
Line 22:4 error: Assignment with incompatible types.
Line 22:11 error: Operator 'or' with incompatible types.
Line 22:15 error: Operator '>' with incompatible types.
Line 27:6 error: 'any-cast' requires an expression of type 'any'.
Line 27:37 error: Operator '+' with incompatible types.
Line 29:6 error: 'any-cast' requires an expression of type 'any'.
Line 29:21 error: The number of parameters in the call to 'g' does not match.
Line 29:32 error: 'any-cast' requires an expression of type 'any'.
Line 31:47 error: Operator 'not' with incompatible types.
Line 33:7 error: Assignment with incompatible types.
Line 33:9 error: 'any-cast' requires an expression of type 'any'.
Line 33:39 error: Operator 'and' with incompatible types.
```

Joc de proves 3 (1 punts). Seguidament decorarem el resultat de `anycast` amb el tipus `T` adequat. Observeu que el node `<T>` no ha estat processat pel *SymbolsVisitor* i per tant no està decorat encara.

Així, el joc de proves:

```
1 func f(a:int, b:bool, x:array[10] of char) : any
2   if a > 10 then
3     return b or not z;
4   else if not b then
5     return x[a];
6   else
7     return a*2.0 + (x!=1);
8   endif
9 endif
10 endfunc
11
12 func main()
13   var i,j : int
14   var x,z : float
15   var a,b : array [10] of char
16   var p : bool
17   i = 5;
18   j = a[i] or b>0;
19   z = i+j;
20
21   z = anycast<float>(f(i+1, z<22, a)) or not p;
22   i = anycast<float>(f(j+2, z<50) + 5.2) + j/i;
23   if j>i or anycast<int>(f(15, false, a)) and not b then
24     b = anycast<char>(f(j, j>=100, b));
25     a[3] = anycast<bool>(f(j, j>=100, b) and not p);
26   endif
27   f(5, true, b);
28   write z;
29 endfunc
```

genera els errors:

```
Line 3:21 error: Identifier 'z' is undeclared.
Line 7:23 error: Operator '+' with incompatible types.
Line 7:27 error: Operator '!=' with incompatible types.
Line 18:4 error: Assignment with incompatible types.
Line 18:11 error: Operator 'or' with incompatible types.
Line 18:15 error: Operator '>' with incompatible types.
Line 21:4 error: Assignment with incompatible types.
Line 21:38 error: Operator 'or' with incompatible types.
Line 22:4 error: Assignment with incompatible types.
Line 22:6 error: 'any-cast' requires an expression of type 'any'.
Line 22:21 error: The number of parameters in the call to 'f' does not match.
Line 22:34 error: Operator '+' with incompatible types.
Line 23:42 error: Operator 'and' with incompatible types.
Line 23:46 error: Operator 'not' with incompatible types.
Line 24:7 error: Assignment with incompatible types.
Line 25:10 error: Assignment with incompatible types.
Line 25:12 error: 'any-cast' requires an expression of type 'any'.
Line 25:42 error: Operator 'and' with incompatible types.
```

2 Arrays multidimensionals (4 punts)

El segon exercici consisteix en estendre els arrays de ASL a arrays multidimensionals. Així podríem escriure per exemple:

```
1 func main()
2   var b : array [10] of float
3   var a : array [6][7][10] of float
4   var i,j,k: int
5
6   i = 0;
7   while i<6 do
8     j = 0;
9     while j<7 do
10      a[i][j] = b;
11      k = 0;
12      while k<10 do
13        write a[i][j][k];
14        write b[k];
15      endwhile
16    endwhile
17  endwhile
18
19  a[2][3][4] = b[5];
20  a[2][6][7] = b[0] + a[1][2][3]/2;
21 endfunc
```

Joc de proves 4 (1 punts). El primer que cal fer es modificar la gramàtica per permetre la declaració d'arrays multidimensionals, acceptant una o mes dimensions. També haureu de modificar el *SymbolsVisitor* perquè crei el tipus adequat.

P.ex. per a la declaració `array [10][20][30] of int` cal crear un tipus array de 10 elements, els quals són arrays de 20 elements, que al seu torn són arrays de 30 enters. Es recomana gestionar les dimensions de dreta a esquerra (e.g. crear primer `array<30,int>`, seguidament usar el tipus obtingut per crear `array<20,array<30,int>>`, i finalment usar el tipus obtingut en el darrer pas per crear `array<10,array<20,array<30,int>>>`).

El mòdul *TypesMgr* també suporta arrays amb elements que no siguin de tipus bàsic.

En aquest joc de proves no apareixen encara accessos a arrays. Es pot superar només declarant-les correctament.

El següent joc de proves:

```
1 func main()
2   var a,b : array [10] of int
3   var p,q : array [6][7][10] of float
4   var x,y : array [6][7][10] of int
5   var i,j,k: int
6
7   p = x;
8   q = p;
9   if i>p or not j then
10      a = x;
11      x = y;
12      b = q;
13   else
14      q = j + 1;
15      i = p + q;
16      i = 0;
17   endif
18 endfunc
```

ha de produir els errors:

```
Line 7:4 error: Assignment with incompatible types.
Line 9:6 error: Operator '>' with incompatible types.
Line 9:12 error: Operator 'not' with incompatible types.
Line 10:7 error: Assignment with incompatible types.
Line 12:7 error: Assignment with incompatible types.
Line 14:7 error: Assignment with incompatible types.
Line 15:11 error: Operator '+' with incompatible types.
```

Joc de proves 5 (1 punts). Seguidament, afegirem els accessos a arrays multidimensionals. Heu de modificar l'accés a arrays tant a *expr* com a *left_expr* per tal de permetre la repetició de dimensions.

Es recomana gestionar la repetició a través de recursivitat per l'esquerra (tant a *expr* com a *left_expr*), cosa que facilitarà molt la decoració dels tipus resultants de cada accés successiu.

Aquest joc de proves:

```

1 func main()
2   var a,b : array [10] of int
3   var p,q : array [6][7][10] of float
4   var x,y : array [6][7][10] of int
5   var z : array [10][7][6] of int
6   var t : array [6] of int
7   var i,j,k: int
8
9   i = 0;
10  while i<6 do
11    j = 0;
12    while j<7 do
13      a[i][j] = b;
14      a[i][j][0] = b[3] + a[0][0][j]*4;
15      b = a[i][j];
16      k = 0;
17      while k<10 do
18        write a[i][j][k] or k!=5;
19        write b[k] + 1;
20        a[k] = b;
21      endwhile
22      a[i] = b[j] * b[i-1];
23    endwhile
24  endwhile
25
26  a[2][3][4] = b[5] > a[1][2][3] or b[1]<a[2][3];
27  a[2][6][7] = b[0] + a[1][2][3]/2;
28
29  i = p[0][0][0] + 5;
30  p[0][0][0] = '2';
31  x[i][j] = y[j][i];
32  x[i][j] = b;
33  b = z[i][j];
34  x[i][j] = t;
35 endfunc

```

produirà els errors:

```

Line 13:8 error: Array access to a non array operand.
Line 14:8 error: Array access to a non array operand.
Line 14:28 error: Array access to a non array operand.
Line 15:12 error: Array access to a non array operand.
Line 18:17 error: Array access to a non array operand.
Line 20:16 error: Assignment with incompatible types.
Line 26:2 error: Array access to a non array operand.
Line 26:22 error: Array access to a non array operand.
Line 26:41 error: Array access to a non array operand.
Line 27:2 error: Array access to a non array operand.
Line 27:22 error: Array access to a non array operand.
Line 29:4 error: Assignment with incompatible types.
Line 30:13 error: Assignment with incompatible types.
Line 33:4 error: Assignment with incompatible types.
Line 34:10 error: Assignment with incompatible types.

```


Joc de proves 6 (0.5 punts). Seguidament, afegirem als nous arrays multidimensionals de ASL la possibilitat de fer *slices* a l'estil de Python, és a dir, un accés a un array que retorna un subarray.

Per exemple, si tenim `var a : array [10] of int`, l'accés `a[4:6]` retorna un array de 3 enters (`array<3,int>`), corresponent a les posicions 4, 5 i 6 de `a`.

Els *slices* només accepten valors enters **constants** i comencen a la posició 0. Per simplificar l'enunciat, els *slices* **no** apareixeran en una *left_expr*. Es recomana usar una subregla **separada** de la de l'accés a array normal. L'operador d'*slice* té, al igual que el d'accés a array, **major prioritat** que la resta d'operadors.

Cal comprovar que el *slice* resultant té almenys un element (es a dir, que el primer valor es menor o igual que el segon) i que els límits del *slice* no excedeixen la mida de l'array. En cas de límits incorrectes, el *slice* tindrà la mida de l'array original. Trobareu els errors necessaris al *SemErrors* modificat.

En el primer joc de proves, afegirem *slicing* només a arrays d'una dimensió.

Aquest joc de proves:

```
1 func main()
2   var a : array [10] of int
3   var b,c : array [5] of int
4   var p,q : array [6][7][5] of float
5   var i,j,k: int
6
7   i = 0;
8   while i<6 do
9     j = 0;
10    while j<7 do
11      p[i][j] = b;
12      q[i-1][j+2] = b[3:7];
13      if i != b[2:6][j] then
14        k = a[5:2][j+1];
15        b = a[5:2];
16      else
17        b = a[8:12];
18        b = a[2:2];
19        j = b[2:2][0];
20      endif
21    endwhile
22  endwhile
23
24  a[2][3][4] = b[4:5][k] > a[1][2][3] or b[1]<a[2][3];
25 endfunc
```

genera els errors:

```
Line 11:16 error: Assignment with incompatible types.
Line 12:20 error: Assignment with incompatible types.
Line 12:22 error: Array slice with invalid limits.
Line 13:9 error: Array slice with invalid limits.
Line 14:9 error: Array slice with invalid limits.
Line 15:7 error: Assignment with incompatible types.
Line 15:9 error: Array slice with invalid limits.
Line 17:7 error: Assignment with incompatible types.
Line 17:9 error: Array slice with invalid limits.
Line 18:7 error: Assignment with incompatible types.
Line 24:2 error: Array access to a non array operand.
Line 24:15 error: Array slice with invalid limits.
Line 24:27 error: Array access to a non array operand.
Line 24:46 error: Array access to a non array operand.
```

Joc de proves 7 (0.5 punts). A continuació, afegirem els casos en que es fa un *slice* en l'última dimensió d'un array multidimensional (observeu que l'última dimensió sempre és un array d'una dimensió). Segons com heu fet els canvis per passar el joc de proves anterior, es possible que aquest us funcioni directament.

Passarem aquest joc de proves:

```

1 func main()
2   var a : array [10][10] of int
3   var b,c : array [5] of int
4   var p,q : array [6][7][8] of float
5   var i,j,k: int
6
7   i = 0;
8   while i<6 do
9     j = 0;
10    while j<7 do
11      p[i][j] = b;
12      p[i+1][j-2] = b[1:8];
13      q[i-1][j+2] = a[j][3:5];
14      q[i-1][j+2] = a[j][0:7];
15      if k+j != a[i][j][k] then
16        c = p[i][j][1:3];
17        k = a[5][j+1];
18        b = a[5:2];
19      else
20        b = a[8:12];
21        b = a[2:6];
22        j = b[0][2:2];
23      endif
24    endwhile
25  endwhile
26
27  a[2][3][4] = b[4:5][k] > a[1][2][3] or b[1] < a[2][3];
28 endfunc

```

generant els errors:

```

Line 11:16 error: Assignment with incompatible types.
Line 12:20 error: Assignment with incompatible types.
Line 12:22 error: Array slice with invalid limits.
Line 13:20 error: Assignment with incompatible types.
Line 14:20 error: Assignment with incompatible types.
Line 15:11 error: Array access to a non array operand.
Line 16:7 error: Assignment with incompatible types.
Line 18:7 error: Assignment with incompatible types.
Line 18:9 error: Array slice with invalid limits.
Line 20:7 error: Assignment with incompatible types.
Line 20:9 error: Array slice with invalid limits.
Line 21:7 error: Assignment with incompatible types.
Line 22:9 error: Array slice in a non array operand.
Line 27:2 error: Array access to a non array operand.
Line 27:15 error: Array slice with invalid limits.
Line 27:27 error: Array access to a non array operand.

```

Joc de proves 8 (1 punt). Finalment, permetrem usar *slices* a qualsevol dimensió.

Observeu que un accés normal a un array es decora amb el tipus dels elements de l'array, mentre que un accés amb un *slice* resulta en un array amb elements del mateix tipus que l'original, pero amb la mida reduïda. Si teniu subregles amb *rule labels* diferents per a cada cas, serà mes fàcil superar aquest darrer joc de proves.

Amb això, el programa següent:

```
1 func main()
2   var a : array [10][10] of int
3   var b,c : array [5] of int
4   var p,q : array [6][7][8] of float
5   var i,j,k: int
6
7   i = 0;
8   while i<6 do
9     j = 0;
10    while j<7 do
11      p[i][j] = b;
12      p[i+1][j-2] = b[1:8];
13      q[i] = a[0:7][0:8];
14      p[j] = a[1:3][2:5];
15
16      q[i-1][j+2] = a[4:6][0:7];
17      if j>a[2:4][k] or a[i][3:7][k]!=p[i][j][k+1] then
18        c = p[i][1:6][1:3];
19        k = a[5:2][j+1];
20        b = a[i+1][2:6];
21      else
22        b = p[8:12][k];
23        b = a[9:11][3:6][i-1][0:4];
24        j = b[2:2][0];
25      endif
26    endwhile
27  endwhile
28
29  a[2][3][4] = b[4:5][k] > a[1][2][3] or b[1] < a[2][3];
30  write p[i][0:3][j+1][0:4][0] == 'c';
31 endfunc
```

ha de donar la sortida:

```
Line 11:16 error: Assignment with incompatible types.
Line 12:20 error: Assignment with incompatible types.
Line 12:22 error: Array slice with invalid limits.
Line 13:13 error: Assignment with incompatible types.
Line 13:15 error: Array slice with invalid limits.
Line 14:6 error: Assignment with incompatible types.
Line 14:8 error: Array slice with invalid limits.
Line 16:20 error: Assignment with incompatible types.
Line 16:22 error: Array slice with invalid limits.
Line 17:5 error: Operator '>' with incompatible types.
Line 18:13 error: Assignment with incompatible types.
Line 19:13 error: Assignment with incompatible types.
Line 19:15 error: Array slice with invalid limits.
Line 22:6 error: Assignment with incompatible types.
Line 22:8 error: Array slice with invalid limits.
Line 23:15 error: Array slice with invalid limits.
Line 29:2 error: Array access to a non array operand.
Line 29:15 error: Array slice with invalid limits.
Line 29:27 error: Array access to a non array operand.
Line 30:31 error: Operator '==' with incompatible types.
```

Informació important

FITXERS PER A L'EXAMEN: Al Racó (`examens.fib.upc.edu`) trobareu un fitxer `examen.tgz` amb el següent contingut:

- `parcial-lab-CL-2025.pdf`: Aquest document, amb l'enunciat i les instruccions.
- `jps`: Subdirectori amb jocs de proves (`jp_chkt_XX.asl`), i la seva corresponent sortida esperada (`jp_chkt_XX.err`).
- `common`: Subdirectori amb els mòduls auxiliars `SemErrors` i `TypesMgr` ampliat amb el codi necessari per a l'examen.
- `evalua.sh`: Script que executa tots els jocs de proves i diu si se superen o no.
- `empaqueta.sh`: Script que crea un fitxer `examen-nom.cognom.tgz` amb la vostra solució. Aquest és el fitxer que cal pujar al Racó.

PASSOS A SEGUIR:

- Feu una còpia de les carpetes `asl` i `common` de la vostra pràctica a un nou directori `examen`.

```
mkdir examen
cp -r practica/asl practica/common examen/
```

- Canvieu al nou directori `examen`, i descomprimiu-hi el fitxer `examen.tgz` del Racó:

```
cd examen
tar -xzf examen.tgz
```

Això extraurà el contingut del paquet, **afegint** al vostre directori `examen` els fitxers llistats anteriorment.

IMPORTANT: Feu-ho en l'ordre especificat (primer una còpia de la vostra pràctica i després descomprimir el `.tgz`). Fer-ho en l'ordre invers causarà que us falti codi necessari a `common` i que els JPs no siguin els adequats.

- Trebal·leu normalment a la carpeta `examen/asl`.

```
cd asl
make antlr
make -j4
...
```

(Si la compilació és lenta per sobrecàrrega del servidor de disc, podeu executar l'script `fast-make.sh`, que fa una còpia temporal i compila al disc local)

- Per veure les diferències entre la sortida del vostre `asl` i la sortida esperada en un joc de proves concret de type check, podeu fer:

```
./asl ../jps/jp_chkt_XX.asl | diff -y - ../jps/jp_chkt_XX.err
```

(Podeu ignorar la línia “There are semantic errors: no code generated” que genera el main)

- Per executar tots els jocs de proves i veure si els passeu, executeu `../evalua.sh`.
- Executeu `../empaqueta.sh` per crear el fitxer d'entrega `../examen-USERNAME.tgz` que cal pujar al Racó.
Els paquets creats sense usar aquest script seran qualificats com **NO PRESENTAT**.